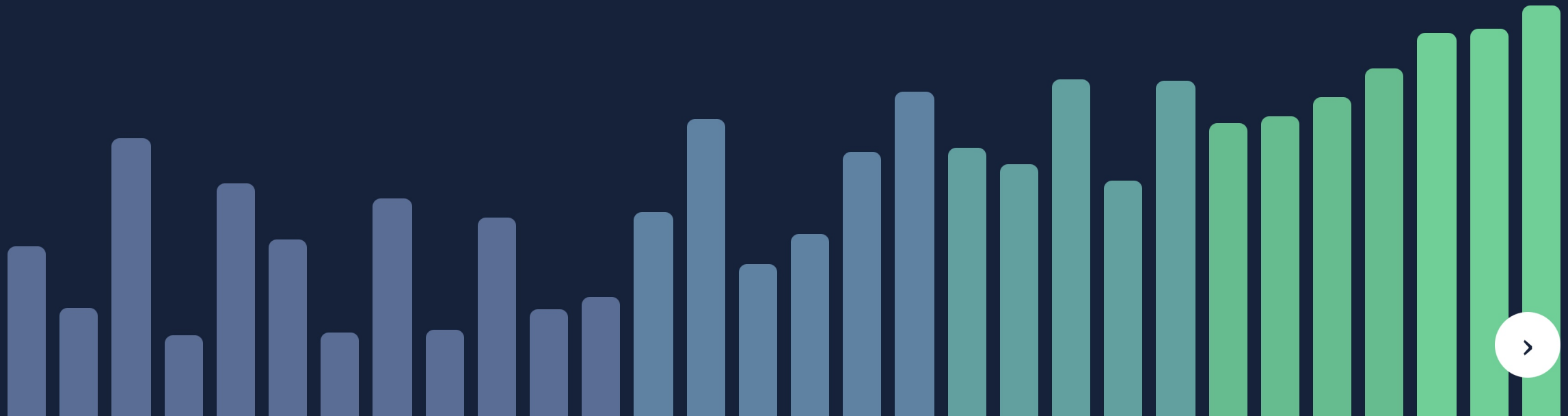
СТРУКТУРИ ОТ ДАННИ И АЛГОРИТМИ

Алгоритми за сортиране

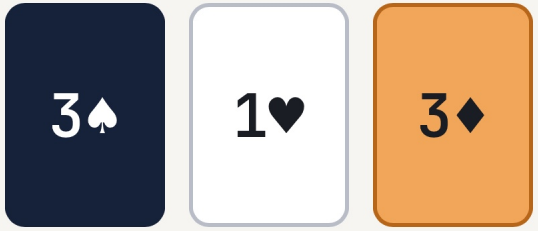
Семинар №2



Стабилност

Алгоритъмът е **стабилен**, ако равните елементи запазват относителната си наредба.

Вход



Сортираме по стойност. Двете тройки са равни, но различими по боя.

Стабилно



3♠ остава пред 3♦, както във входа.

Нестабилно (възможно)



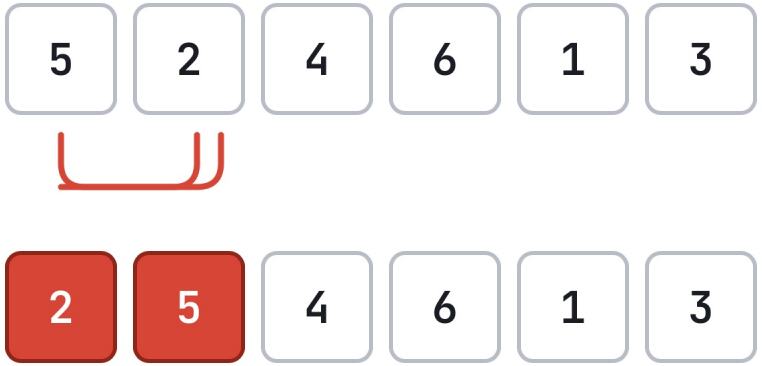
3♦ се оказва пред 3♠: редът на равните е нарушен.

In-place

Алгоритъмът не използва допълнителна памет, зависеща от входа: **O(1)**.

Bubble Sort

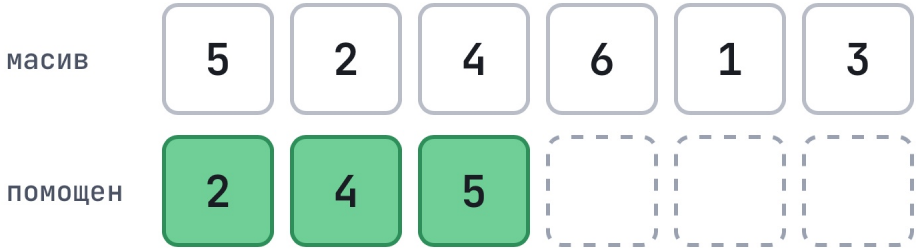
In-place · O(1)



Размените стават в същия масив. Освен няколко променливи, не е нужна друга памет.

Merge Sort

Не е in-place · O(n)



Сливането записва в помощен масив със същия размер, затова паметта расте с n.

Bubble Sort: идея

сравнение · размяна · сортирано

На всеки пас съседните елементи се сравняват и при нужда се разменят. Най-големият „изплува“ в края.



сравняваме 5 и 2

5 > 2 → размяна

5 > 4 → размяна

5 < 6 → без размяна

6 > 1 → размяна

6 > 3 → размяна, 6 е на мястото си

Bubble Sort: код и сложност

```cpp
template<class T>
void bubbleSort(T* arr, unsigned length)
{
    size_t lastSwappedIndex = length - 1;

    for (size_t i = 0; i < length; i++)
    {
        unsigned lastSwappedIndexTemp = lastSwappedIndex;
        for (size_t j = 0; j < lastSwappedIndex; j++)
        {
            if(arr[j] > arr[j + 1])
            {
                std::swap(arr[j], arr[j + 1]);
                lastSwappedIndexTemp = j;
            }
        }

        if(lastSwappedIndex == lastSwappedIndexTemp) { break; }
        lastSwappedIndex = lastSwappedIndexTemp;
    }
}
```

Най-добър случай

O(n)

сортиран масив: нито една размяна

Среден случай

O(n²)

Най-лош случай

O(n²)

Памет

O(1)

Стабилен

In-place

`lastSwappedIndex` пази мястото на последната размяна. Всичко вдясно от него вече е подредено, затова следващият пас стига само дотам. Пас без размяна прекъсва цикъла.

Insertion Sort




```cpp
template<class T>
void insertionSort(T* arr, unsigned length)
{
    for (size_t i = 1; i < length; i++)
    {
        T elementToInsert = arr[i];
        int j = i - 1;
        while (j >= 0 && arr[j] > elementToInsert)
        {
            arr[j + 1] = arr[j];
            --j;
        }
        arr[j + 1] = elementToInsert;
    }
}
```

Като подреждане на карти в ръка: всеки нов елемент се вмъква на мястото си в сортираната лява част.

най-добър / среден / най-лош	памет		
$O(n)$ / $O(n^2)$ / $O(n^2)$	$O(1)$	Стабилен	In-place

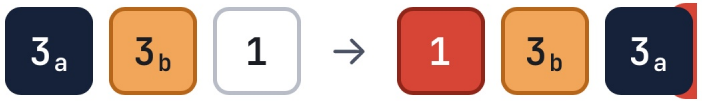
Полезен за малки или почти сортирани масиви.

Selection Sort

Намира се минимумът в несортираната част и се разменя с първия ѝ елемент.



КОНТРАПРИМЕР ЗА НЕСТАБИЛНОСТ



Минимумът 1 се разменя с първата тройка и 3_a прескача 3_b.

```
template<class T>
void selectionSort(T* arr, unsigned lenght)
{
    for (size_t i = 0; i < lenght; i++)
    {
        size_t minElementIndex = i;
        for (size_t j = i + 1; j < lenght; j++)
        {
            if(arr[j] < arr[minElementIndex])
            {
                minElementIndex = j;
            }
        }

        if(i ≠ minElementIndex)
        {
            std::swap(arr[i], arr[minElementIndex]);
        }
    }
}
```

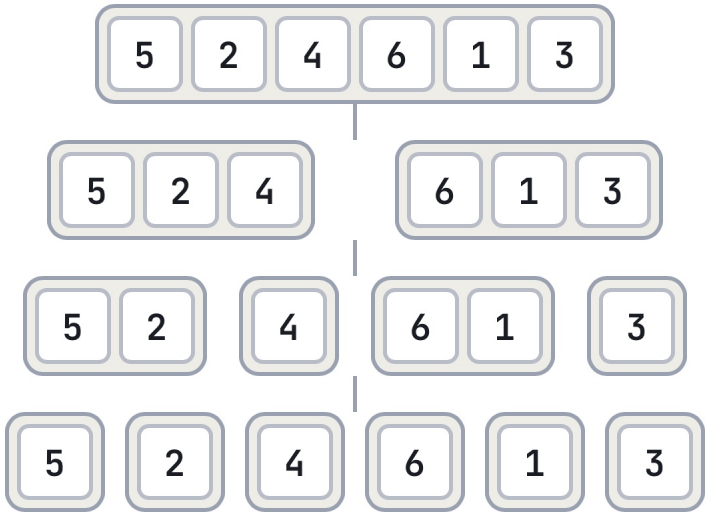
време, винаги	памет		
$O(n^2)$	$O(1)$	In-place	Нестабилен

Полезен, когато записите са скъпи: най-много n−1 размени.

Merge Sort

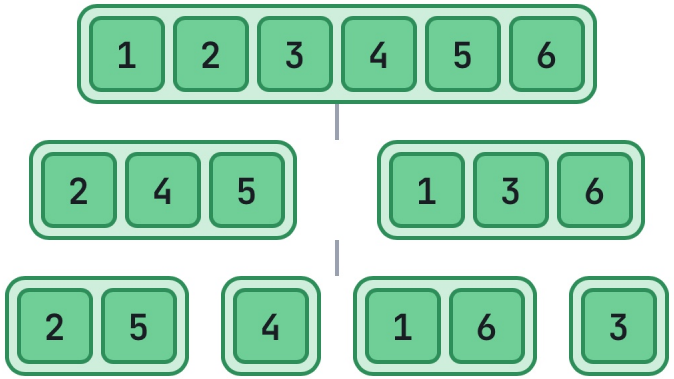
Разделяй и владей: масивът се дели наполовина, двете половини се сортират рекурсивно и се сливат.

РАЗДЕЛЯНЕ ↓



Делим, докато остане по един елемент.

СЛИВАНЕ ↑



Две сортирани части се сливат в една: [2, 4, 5] и [1, 3, 6] дават [1, 2, 3, 4, 5, 6].

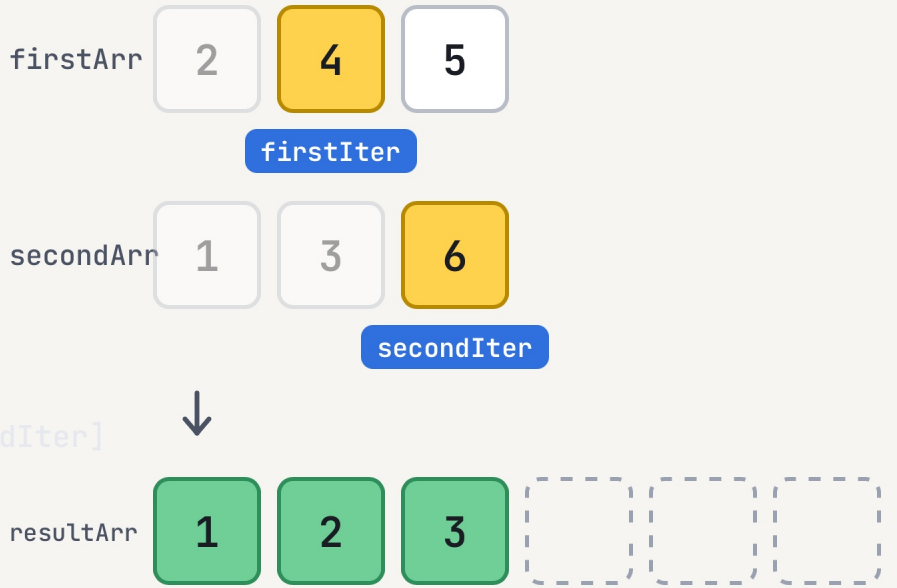
Merge Sort: функцията Merge

```
template <class T>
void Merge(T* firstArr, size_t firstSize,
           T* secondArr, size_t secondSize, T* resultArr)
{
    size_t resultIter = 0;
    size_t firstIter = 0;
    size_t secondIter = 0;

    while(firstIter < firstSize && secondIter < secondSize)
        resultArr[resultIter++] = (firstArr[firstIter] ⩽ secondArr[secondIter]
            ? firstArr[firstIter++] : secondArr[secondIter++]);

    while(firstIter < firstSize)
        resultArr[resultIter++] = firstArr[firstIter++];

    while(secondIter < secondSize)
        resultArr[resultIter++] = secondArr[secondIter++];
}
```



По-малкият елемент под двата итератора отива в resultArr. Следва сравнението на 4 и 6.

време, във всички случаи	памет			
$O(n \log n)$	$O(n)$	Не е in-place	Стабилен	Със $\leqslant$ при равни елементи се взема от първия масив, затова редът им се запазва.

Merge Sort: рекурсията

```cpp
template <class T>
void MergeSortStep(T* pArr, size_t Size, T* pBuffer)
{
    if(Size ⩽ 1)
        return;

    size_t middle = Size / 2;

    MergeSortStep(pArr, middle, pBuffer);

    MergeSortStep(pArr + middle, Size - middle, pBuffer + middle);

    Merge(pArr, middle, pArr + middle, Size - middle, pBuffer);

    for(size_t i = 0; i < Size; i++)
        pArr[i] = pBuffer[i];
}
```

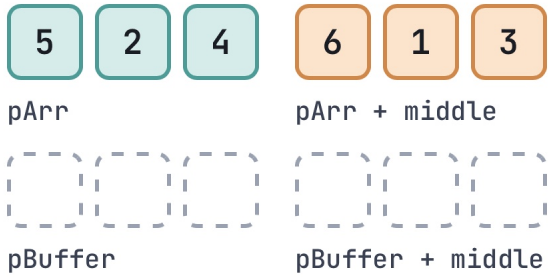
```cpp
template <class T>
void MergeSort(T* pArr, size_t Size)
{
    if (!pArr || Size == 0)
        return;

    T* pBuffer = new T[Size];

    MergeSortStep(pArr, Size, pBuffer);

    delete[] pBuffer;
}
```

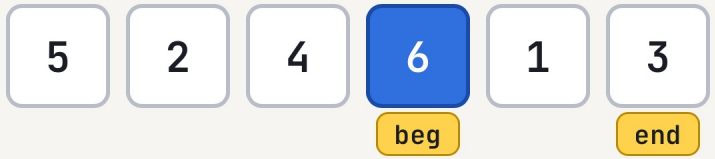
ЕДИН БУФЕР ЗА ЦЯЛОТО ДЪРВО · SIZE = 6, MIDDLE = 3



Quick Sort: идея

Избира се **pivot**. Partition го поставя на крайното му място: вляво са по-малките, вдясно по-големите. Двете части се сортират рекурсивно.

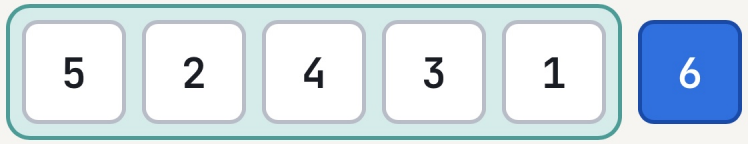
1 · pivot = arr[size / 2] = 6



2 · beg спира при 6, end при 3: размяна



3 · beg стига до end → край, връща 5



1. **`partition`** връща индекса на pivot-а. Там той остава завинаги.
2. Рекурсия: **`quickSort(arr, pIdx)`** за лявата част и **`quickSort(arr + pIdx + 1, size - pIdx - 1)`** за дясната.
3. Тук 6 е най-голямото, затова дясната част е празна. Остава да се сортира [5, 2, 4, 3, 1].

Quick Sort: код

```cpp
template <class T>
int partition(T* arr, size_t size) {
    T pivot = arr[size / 2];
    int beg = 0, end = size - 1;

    while (true) {
        while (arr[beg] < pivot) {
            beg++;
        }

        while (arr[end] > pivot)
            end--;

        if (beg >= end)
            break;

        if (arr[beg] == arr[end])
            beg++;

        std::swap(arr[beg], arr[end]);
    }
    return beg;
}
```

```cpp
template <class T>
void quickSort(T* arr, size_t size) {
    if (size <= 1)
        return;
    int pIdx = partition(arr, size);
    quickSort(arr, pIdx);
    quickSort(arr + pIdx + 1, size - pIdx - 1);
}
```

1. Указателите **beg** и **end** се срещат при мястото на pivot-а, затова **`return beg`**.
2. Проверката **`arr[beg] == arr[end]`** предпазва от безкраен цикъл при повтарящи се елементи.
3. Двете рекурсивни извиквания работят върху подмасиви чрез указателна аритметика, без допълнителна памет.

Quick Sort: сложност и най-лош случай

най-добър и среден
O(n log n)

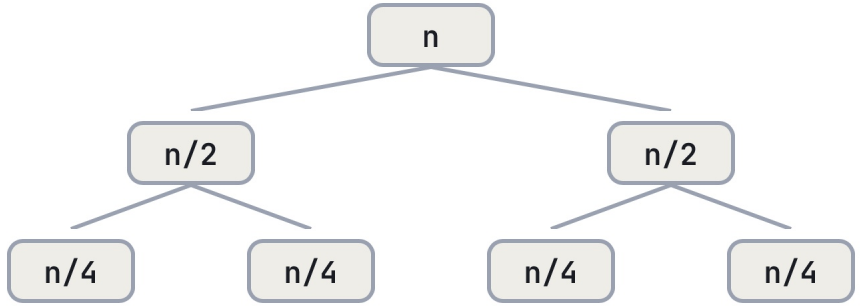
най-лош
O(n²)
pivot е винаги най-малък или най-голям

памет
O(log n)
средно, заради стека на рекурсията

In-place

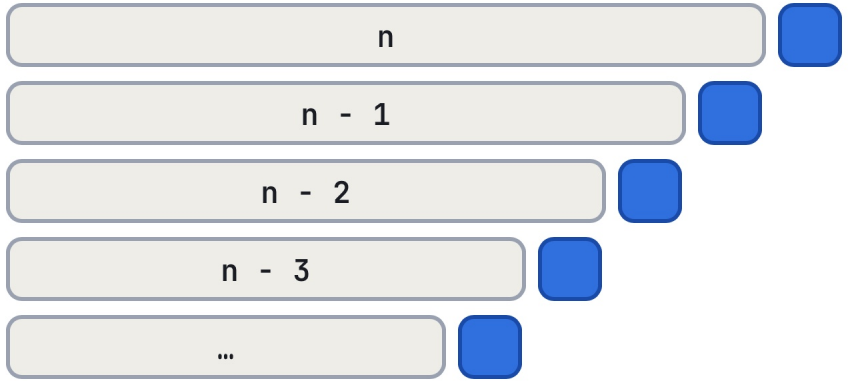
Не е стабилен

БАЛАНСИРАНО РАЗДЕЛЯНЕ · ДЪЛБОЧИНА ≈ LOG N



Всяко ниво струва O(n), а нивата са log n.

ИЗРОДЕНО РАЗДЕЛЯНЕ · ДЪЛБОЧИНА ≈ N

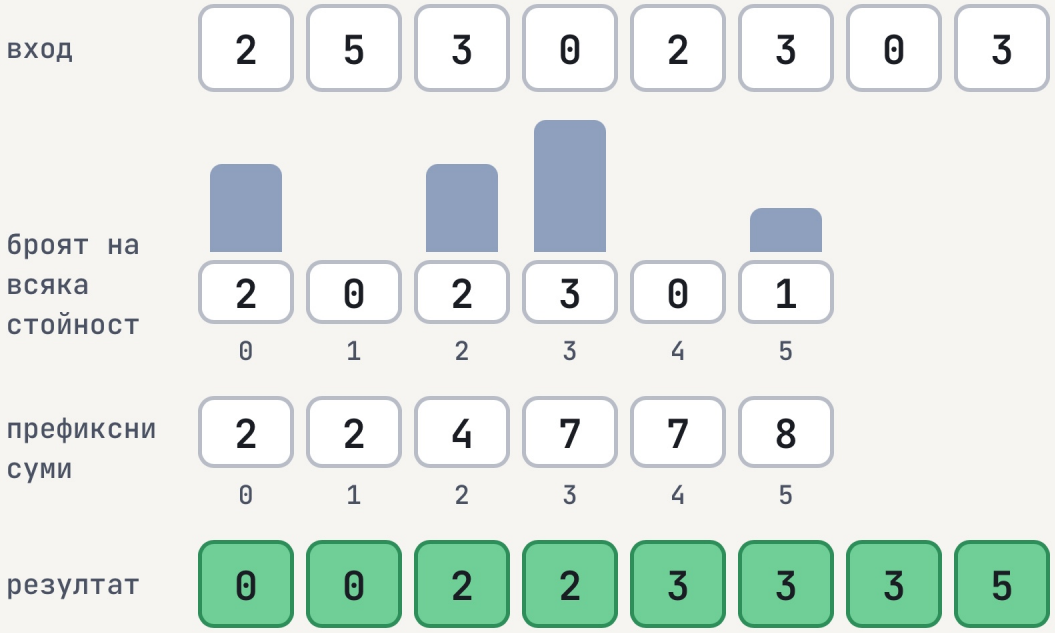


Pivot-ът е най-голям или най-малък, остава една голяма част: n + (n−1) + … = O(n²).

Най-лош случай: pivot-ът е всеки път най-малкият или най-големият елемент. **Изродяване:** разделянето дава части n−1 и 0, дървото на рекурсията става „стълба“ с дълбочина n и времето нараства до O(n²). **Избягва се** със случаен pivot или median-of-three.

Counting Sort

Сортиране **без сравнения**: броят се срещанията на всяка стойност. Пример с k = 5, различен от останалите.



време	памет
O(n + k)	O(k)

Стабилен **Не е in-place**

Стабилен е, когато резултатът се попълва с обхождане отзад напред.

Кога е полезен: малък диапазон на стойностите.

Counting Sort: код

```cpp
void counting_sort(int* arr, size_t arrLength) {
    constexpr int MAX_SIZE = 1e5;
    int arr_copy[MAX_SIZE];

    int max = arr[0];
    for (int i = 0; i < arrLength; ++i) {
        if (max < arr[i]) {
            max = arr[i];
        }
        arr_copy[i] = arr[i];
    }

    int count[MAX_SIZE];
    for (int i = 0; i < max + 1; ++i) {
        count[i] = 0;
    }

    for (int i = 0; i < arrLength; ++i) {
        count[arr[i]]++;
    }

    for (int i = 1; i ⩽ max; ++i) {
        count[i] += count[i - 1];
    }

    for (int i = arrLength - 1; i ⩾ 0; --i) {
        arr[count[arr_copy[i]] - 1] = arr_copy[i];
        count[arr_copy[i]]--;
    }
}
```

1. `count[arr[i]]++` брои срещанията; `count[i] += count[i - 1]` ги превръща в префиксни суми.
2. Обхождането отзад напред запазва реда на равните: алгоритъмът е стабилен.
3. Работи за неотрицателни числа, като `max + 1` трябва да е най-много `MAX_SIZE`.

СРАВНЕНИЕ

Обобщителна таблица

АЛГОРИТЪМ	НАЙ-ДОБЪР	СРЕДЕН	НАЙ-ЛОШ	ПАМЕТ	IN-PLACE	СТАБИЛЕН	КОГА Е ЕФЕКТИВЕН
Bubble	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	✓	✓	Малки или почти сортирани масиви
Insertion	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	✓	✓	Малки или почти сортирани масиви
Selection	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	✓	✗	Скъпи записи: най-много n−1 размени
Merge	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	✗	✓	Големи масиви, когато е нужна стабилност
Quick	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	✓	✗	Големи масиви в общия случай
Counting	$O(n + k)$	$O(n + k)$	$O(n + k)$	$O(k)$	✗	✓	Малък диапазон на стойностите

БИБЛИОТЕКА

std::sort

```
#include <algorithm>

template <class RandomIt>
void sort(RandomIt first, RandomIt last);

template <class RandomIt, class Compare>
void sort(RandomIt first, RandomIt last, Compare comp);
```

`first`	начало на диапазона
`last`	елементът след последния
`comp`	критерий за сравнение, по подразбиране `operator<`

ОБИКНОВЕНО СЕ РЕАЛИЗИРА КАТО INTROSORT

Quick Sort
основният алгоритъм

Heap Sort
когато рекурсията стане твърде дълбока: избягва $O(n^2)$

Insertion Sort
за малките подмасиви накрая

O(n log n) · In-place · Не е стабилен

За стабилност има `std::stable_sort`.

БИБЛИОТЕКА

Компаратори

Собствени функции, **std::less** и **std::greater**

```cpp
bool compareLastDigit(int a, int b) {
    return a % 10 < b % 10;
}

std::vector<int> v = {25, 42, 33, 14, 56};

// по последна цифра
std::sort(v.begin(), v.end(), compareLastDigit);

// намаляващ ред
std::sort(v.begin(), v.end(), std::greater<int>());
```

COMPARELASTDIGIT

преди	25	42	33	14	56
след	42	33	14	25	56

STD::GREATER<INT>()

преди	25	42	33	14	56
след	56	42	33	25	14

std::less и **std::greater** са функционални обекти с предефиниран **operator()**.